

Sundsvalls badplatser - Att skapa en mashup

Frontend-baserad webbutveckling

Sundsvalls badplatser
Att skapa en mashup

Josefine Backlund



Mittuniversitetet
MID SWEDEN UNIVERSITY

Sundsvalls badplatser - Att skapa en mashup
Josefine Backlund 2026-03-18

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Josefine Backlund, joba1602@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Sammanfattning

Denna rapport beskriver arbetsprocessen under skapandet av en mashup. Processen delas upp i en planerings- och implementeringsfas. Planeringen innefattar skisser av webbsidan och implementeringen beskriver hur webbsidans utseende och funktionalitet realiserats. Slutsatsen innehåller reflektioner om projektet där det som gått bra och dåligt diskuteras samt det som skulle kunnat göras annorlunda.

Innehållsförteckning

1. Inledning.....	1
1.1 Bakgrund och problemmotivering.....	1
1.2 Övergripande syfte.....	1
1.3 Detaljerad problemformulering.....	1
2. Teori.....	2
3. Metod.....	4
4. Konstruktion.....	5
5. Resultat.....	8
6. Slutsatser.....	9

1. Inledning

1.1 Bakgrund och problemmotivering

Det finns ett oändligt utbud av webbsidor idag, och de flesta reflekterar med stor sannolikhet inte över vad som händer bakom kulisserna medan de använder sig av tjänster på nätet. Vissa av dessa applikationer kan realiseras med hjälp av att kombinera olika tjänster och källor, och på så sätt kan något nytt skapas tack vare olika sorters innehåll som egentligen inte är relaterade till varandra. När dessa källor kommunicerar med varandra skapas något som kallas för en "mashup". Eftersom att man använder tjänster som redan finns färdiga att implementera i sitt eget projekt, krävs det mindre kod för den som utvecklar, alltså kommer utvecklingen gå snabbare. Man kan dessutom skapa tjänster som ger unika resultat, och på så vis underlätta för användare som annars hade behövt hämta sin information på flera olika ställen [1].

Mashups innebär därmed många möjligheter för att uppfylla användarens specifika behov, samtidigt som mindre kod än vanligt krävs, vilket gör att koden också blir lättare att underhålla och vidareutveckla. Detta väcker också frågor gällande hur man framställer en mashup, och mer specifikt hur arbetsprocessen för detta kan se ut.

1.2 Övergripande syfte

Detta projekt har genomförts i syfte att ge en inblick i hur man kan kombinera olika tjänster och källor för att i sin tur skapa en ny webbapplikation. Skapandeprocessen beskrivs via ett konkret exempel på en mashup.

1.3 Detaljerad problemformulering

Målet med detta projekt är att skapa en webbsida som förser användaren med visuell lokalisering av badplatser i Sundsvalls kommun, samt specifik information om varje badplats. Användaren ska kunna:

- Se på en karta var varje badplats är lokaliserad.
- Få ytterligare information om varje badplats bestående av namn, beskrivning av badplatsen samt eventuell temperatur i vattnet och när detta senast kontrollerades.

Skapandet av webbsidan ska inkludera följande:

- En automatiserad arbetsprocess.
- Dokumenterad Javascript-kod.
- Responsivitet.
- Välstrukturerad CSS med hjälp av SASS/SCSS.
- Animeringar som förbättrar användarupplevelsen.

2. Teori

För att minska det manuella arbetet samtidigt som man kan optimera filer, kan man använda sig av en *automatiserad arbetsprocess*. Detta är särskilt användbart för webbutveckling då optimerade webbsidor laddar snabbare, vilket underlättar och förbättrar upplevelsen för både utvecklaren och användaren. Det finns ett antal olika verktyg man kan använda sig av för att automatisera, och ett av dem heter Vite. Vite är ett byggverktyg som har som fokus att skapa snabbare applikationer med en smidigare utvecklingsmiljö. Detta uppnås tack vare en utvecklingsserver med live-uppdatering och ett bygg-kommando som kompilerar filerna. Utvecklaren har tillgång till tre kommandon: `npm run dev` som öppnar en live-server, `npm run build` som optimerar alla filer samt `npm run preview` som ger utvecklaren en överblick över hur den slutgiltiga produktionen kommer att se ut [2]. Detta kan även göra publiceringsprocessen mindre manuell. Ett Github-repository som skapats med ett Vite-projekt kan importeras till en publiceringstjänst, till exempel Netlify, som känner av att Vite används, och kan uppdatera webbsidan efter en merge till main utan att någon manuell publicering behöver göras [3].

För att kunna använda Vite behöver man också NodeJs, som är en JavaScripts-miljö där *NPM*, eller Node Package Manager, finns tillgängligt. Denna tjänst hanterar de olika paket och bibliotek som finns att använda vid utveckling [4]. I samband med en uppstart av ett Vite-projekt kan man installera npm i sitt projekt, och detta gör det i sin tur möjligt att installera NPM-paket. SASS finns tillgängligt som ett sådant paket. SASS är en *CSS-preprocessor*, alltså ett program som gör det möjligt att skriva CSS-kod i annan syntax med olika former av tillägg som inte finns inom traditionell CSS, till exempel mixin eller användning av nesting [5]. Det blir också enklare att strukturera koden då det är möjligt att dela upp koden i flera filer med hjälp av *partials* [6]. Ett annat paket heter Vite Plugin Image Optimizer, som i sin tur behöver Sharp och eventuellt Svgo. Detta plugin kombinerar Sharp och Svgo, och komprimerar bildfiler i olika format till önskad grad enligt konfiguration vid bygg-kommandot i Vite [7].

I syfte att göra kodfiler lättare att sätta sig in i för olika utvecklare kan man dokumentera kodens funktioner på ett särskilt sätt för att automatiskt kunna generera och samla dokumentationen, till exempel med *JSDoc* [8]. Det finns verktyg som hjälper till att skapa en webbsida med den färdiga dokumentationen. Till exempel finns det ett NPM-paket med samma namn som kan installeras för att uppfylla detta syfte [9].

När det kommer till programkod så finns det många potentiella fel som kan uppstå av olika anledningar. Ibland kan det till exempel bero på användaren eller ha att göra med server- eller internetproblem. I sådana fall blir det svårt att styra hur koden kommer att agera beroende på var problemet uppstår. Därför, när man till exempel använder fetch-anrop, är

Sundsvalls badplatser - Att skapa en mashup

Josefine Backlund 2026-03

det användbart att anamma en kodstruktur som består av “try”, “catch” och eventuellt “finally” när man hanterar *promises* [10]. Promises används i asynkrona funktioner och innebär just vad det betyder, ett löfte. Något kommer att hända, men exakt när är oklart. Detta lämpar sig väl vid till exempel AJAX-anrop. Ett promise har tre olika tillstånd, där den inleds med “pending”, alltså att anropet pågår, “resolved” som betyder att det lyckats, eller “rejected” där något gått fel [11]. För att underlätta felhantering kan man som tidigare nämnt använda sig av try/ catch/ finally. Inom try finns det anrop man vill åstadkomma, medan catch innehåller direktiv om vad som ska hända om anropet inte fungerar. Finally är valfritt, men kan vara användbart om man vill att något ska ske oavsett om anropet lyckas eller inte [10].

Det finns olika saker som påverkar användarupplevelsen, som till exempel användbarheten och laddningstid, men något som också kan höja intrycket på en webbsida är *animationer*. Med animationer kan man gradvis förändra ett element, till exempel med färg eller position. Det finns olika sätt att göra detta på, och ett sätt är att använda sig av transitions. Transitions är något begränsade och lämpar sig för användarinteraktion som till exempel hover. Det kan användas för att manipulera element för att uppnå en smidigare övergång från ett stadie till ett annat, men det finns bara två stadier: start och slut [12]. Transform är också användbart för att till exempel kunna rotera, förflytta, skala om eller förvränga element [13]. För att kunna manipulera element på olika sätt under flera olika stadier kan man använda sig av keyframes. En animation skriven med hjälp av keyframes behöver ett namn och ett antal regler. Sedan kan denna animation inkluderas i CSS-koden där den ska ta effekt tillsammans med hur länge animationen ska pågå [14]. Här finns möjlighet att påverka till exempel upprepning, hur animeringen ska spelas upp eller om den ska vara pausad eller aktiv [15].

När en webbsida har innehåll som berör till exempel lokalisering, kan det vara användbart att implementera *kartor* i syfte att ge användaren en överblick över innehåll som kopplas till flera olika platser. Detta är vanligt i applikationer som informerar om väderleksrapporter till exempel. Som utvecklare finns det olika tjänster att använda som uppfyller detta syfte samt ger möjligheten att manipulera lager och markörer för att anpassa till specifika projekt. Leaflet är en sådan tjänst, och är ett JavaScript-bibliotek som erbjuder interaktiva kartor med många kod-exempel och instruktioner på deras webbsida [16].

3. Metod

Till att börja med kommer arbetet att kräva en planering, vilket inkluderar en moodboard tillsammans med skisser för mobil och desktop som skapats i programmet Figma. Projektets moodboard kommer att innehålla de färger och fonter som ska användas.

Nästa steg består av att påbörja ett Vite-projekt. Nödvändiga plugins kommer att installeras för att kunna nyttja den automatiserade arbetsprocessen. Dessa plugins kommer att justeras i en fil för att anpassas efter projektets behov. När detta slutförts kommer ett Git-repository skapas och kopplas till Github, och en branch utöver main kommer att läggas till i syfte för utveckling. Detta projekt kommer att importeras till Netlify och automatiskt uppdateras vid hantering i Github.

Efter projektet etablerats och är redo att påbörjas, kommer den statiska webbsidan designas med skisserna från planeringen som utgångspunkt. Sidan ska också anpassas efter olika skärmstorlekar. Härnäst ska det grafiska innehållet skapas eller väljas ut, till exempel bilder, ikoner och logotyp etcetera. Bilder kommer att konverteras till olika format, och även olika storlekar vid behov med hjälp av ett fotoredigeringsprogram, Photopea. Dessa kommer sedan automatiskt att komprimeras tack vare ett plugin i Vite.

När webbsidan ser ut som planerat kommer funktionaliteten att realiseras med hjälp av Vanilla JavaScript. AJAX-anrop kommer att göras mot ett öppet API och kombineras med karttjänsten Leaflet. JSDocs kommer att användas för att dokumentera koden i syfte att strukturera och göra det lättare för utomstående utvecklare att sätta sig in i projektet. I syfte att förhöja användarupplevelsen kommer animationer med ett tydligt syfte implementeras på webbplatsen. Som en sista kontroll kommer webbsidan även att kontrolleras på olika skärmstorlekar och i olika webbläsare.

4. Konstruktion

Moodboarden var det första som fick fokus i arbetet. Färgerna valdes ut i hänsyn till vatten i syfte att skapa ett lämpligt tema för badplatser. Figma-skisser skapades sedan, där mobildesignen (se Bilaga A) skapades först och sedan anpassades för större skärmar med ett desktop-exempel (se Bilaga B). Ett Vite-projekt inleddes med Vanilla JavaScript som alternativ.

De NPM-paket som installerades var SASS, Vite Plugin Image Optimizer, Sharp, Svgo och JSDocs. För att kunna konfigurera vissa av dessa plugins skapades en fil vid namn "vite.config.js". I denna fil justerades komprimeringsgraden för olika bildformat, så att png- och jpg-filer komprimerades till 80 procent och webp-filer till 70 procent. En importerad funktion från Vite, rollupOptions, vars huvudsakliga syfte är att instruera Vite om att det finns fler html-sidor som ska byggas, gjorde det även möjligt att justera domännamnets ändelse.

Efter projektet inleddes skapades ett Git-repository av filerna länkat till Github. Ytterligare en gren, "development", skapades för att kunna nyttja en utvecklingsmiljö som kan kontrolleras innan den förenas med main. Detta projekt importerades från Github till webbtjänsten Netlify, för att automatiskt publicera önskat innehåll.

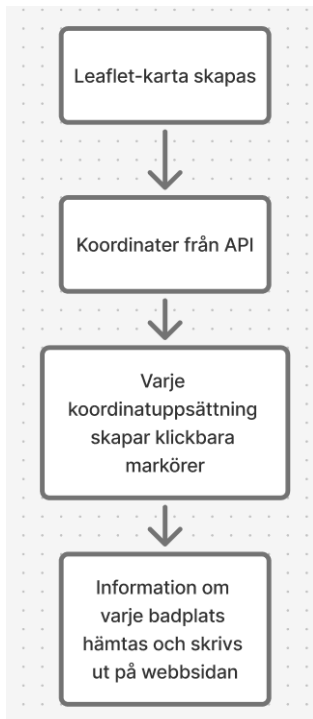
Ikonerna valdes även ut från Google Icons och lades in i skisserna för att förbättra användarupplevelsen med tydligare information. En bild på en av badplatserna inom kommunen inkluderades och konverterades till webp och jpg i två olika storlekar för större och mindre skärmar i syfte att optimera det grafiska innehållet.

När det kommer till designen av webbplatsen lades statiska exempel in för att den data som senare hämtas ska anamma rätt stilregler. För att strukturera CSS-koden användes SASS som installerades tidigare. En mapp rörande designen skapades med en huvudsaklig SASS-fil och ett antal partials för att underlätta utvecklingsprocessen.

När alla element följde skissernas utformning togs det statiska innehållet bort för att istället hämta relevant data med AJAX-anrop från det öppna API:et med hjälp av JavaScript. En karta skapades med Leaflet, och den data som hämtades användes för att placera ut markörer på kartan för alla badplatser, se Figur 1.1.

Sundsvalls badplatser - Att skapa en mashup

Josefine Backlund 2026-03



Figur 1.1 Flödesschema

Eftersom det huvudsakliga innehållet på webbplatsen hämtar data via ett fetch-anrop i en asynkron funktion, användes try/ catch för att få mer kontroll vid oförutsägbara fel. Som Figur 1.2 visar är det då möjligt att generera felmeddelanden till användaren och skriva ut detaljerna i konsolen i de fall anropet ej lyckas. Fördelen med denna metod är att ett kodblock inom finally också blir möjligt, där kod körs oavsett resultatet. I denna del har en laddnings-animation justerats.

```
try {
  const response = await fetch(url);
  const mapData = await response.json();
  placeMarkers(mapData.data);
} catch (error) {
  console.error("Fel: " + error);
  //Skapar felmeddelande om API:et krånglar.
  const errorEl = document.querySelector("#error");
  const errorMessage = document.createElement("p");
  const errorMessageContent = document.createTextNode(
    "Det verkar inte finnas några platser att hämta just nu, försök igen om en liten stund!",
  );
  errorMessage.appendChild(errorMessageContent);
  errorMessage.classList.add("error");
  errorMessage.classList.add("textandicon");
  errorEl.appendChild(errorMessage);
} finally {
  //Justerar klasser för att ta bort laddningsanimation och visa kartan.
  loader.classList.add("hidden");
  mapContent.classList.remove("seethrough");
}
```

Figur 1.2 Try/Catch/Finally

Den viktigaste animationen som skapats för webbplatsen var laddningsanimeringen, som tidigare nämnts. Detta gjordes för att visa användaren att något kommer att hända, men inte är färdigt än. Denna kunde skapas med hjälp av keyframes, som användes på ett tomt div-element med en färg på border, och en annan färg på border-top.

Transition och translate användes också i syfte att förbättra användarupplevelsen, till exempel vid hämtning av information om varje badstrand och vid hover över länken i sidfoten. För informationen justerades opacitet och placering längs Y-axeln, vilket signalerade för användaren att något särskilt pågår, och därmed drar till sig uppmärksamhet. Ikonen som finns bredvid länken ändrar placering vid hover, för att förmedla att länken är klickbar och att användaren kommer skjutas vidare till något annat.

När all programkod var färdig skapades kommentarer i kodfilen enligt JSDocs standard. Objekt och funktionerna som hanterar dessa beskrevs. Med ett kommando i terminalen genererades mappen docs, som innehåller nödvändiga filer för att skapa en webbsida baserat på dokumentationen i koden. Åtkomst till denna webbsida gavs i projektets sidfot, samt i README-filen i Github-repot.

När funktionaliteten av webbsidan kontrollerats anpassades den till mindre skärmar med hjälp av media queries. Ett mörkt tema skapades också i syfte att vara anpassat för system som föredrar detta. Eftersom att webbsidan blir avlång på mindre skärmar, justerades kod och CSS för att användaren ska scrollas ner automatiskt vid klick av varje badplats. Utan denna justering kan användaren lätt missa att informationen skapats under kartan. Webbsidan publicerades, och kontrollerades via Google Chrome, Firefox samt Safari.

5. Resultat

Användaren bemöts av en karta med ett antal markörer utplacerade på olika ställen runtom i Sundsvalls kommun. Kartan är interaktiv och ger möjlighet att klicka på varje markör, och vid varje klick visas information om den specifika platsen som valts ut. Varje badplats har ett namn tillsammans med en beskrivning, och vissa av dem innehåller också temperaturer och när den senaste temperaturen registrerades.

Projektet har skapats med Vite, och har använt ett plugin för att komprimera bilder, vilket optimerar det grafiska innehållet. Publiceringsprocessen har genomförts automatiskt tack vare importering av ett Github-repository till Netlify.

I sidfoten på webbsidan finns en länk till genererad dokumentation gällande programkoden skriven enligt JSDoc-standard.

SASS har installerats och skapat en kodstruktur för CSS som sorterat upp innehållet i olika partials.

Webbsidan anpassar sig efter olika skärmstorlekar genom användandet av media queries.

Det finns några animeringar på webbsidan. Vid långsamt internet eller problem med fetch-anrop visas en laddnings-animation där kartan ska finnas. Vid varje klick på en markör vid kartan visas information under kartan där en transition använts i kombination med translate och byte av klass. Länken i sidfoten reagerar på hover och förflyttar ikonerna upp och ner.

6. Slutsatser

Arbetsprocessen för detta projekt har varit ovanligt smidig, åtminstone om jag jämför med tidigare uppgifter i slutet på andra kurser. Jag är dock rätt säker på att detta har mycket att göra med den automatiserade arbetsprocessen, mer rutin gällande versionshantering, planering samt utförande. Jag finner det också betydligt lättare att navigera i CSS-kod med hjälp av SASS, vilket har varit ett stort problem för mig vid andra tillfällen. Något som jag uppskattade var att fokusera på funktionalitet och kvalitet snarare än kvantitet, och med det menar jag många undersidor och mycket material för innehållet. Detta tillfälle kunde verkligen nyttjas till att skapa något användbart på riktigt, vilket får mig att känna mig som en riktig webbutvecklare.

Det var inget jag tänkte på under produktionen av webbplatsen, men något jag tycker skulle kunna förbättras är att på ett tydligt sätt förmedla till användaren vilken badplats de har valt på kartan och läser information om för tillfället. Det finns ett antal markörer och det kan vara lätt att glömma bort vilken man klickade på.

När det kommer till svårigheter så är det för det första nytt för mig att dokumentera kod på sättet som gjorts i projektet, och det tog lite tid att skriva då jag ofta är osäker på om jag gjort på rätt sätt. Det finns mycket information att hämta på nätet, men jag finner det svårt att applicera det på min situation. Jag hade även problem med API:et jag använde, då det på vissa ställen dök upp till exempel “\\r”, som behövde åtgärdas med justering av utskriften. Ett annat problem med API:et var att anropen var begränsade, vilket förmodligen beror på att det är öppet för alla att använda. Detta kunde vara frustrerande under utvecklingen, och skulle jag arbeta med ett öppet API igen skulle jag nog spara det lokalt under tiden jag justerar utskrifterna istället för att behöva vänta på att kunna göra anrop igen efter 30 sekunder.

Källförteckning

- [1] GeeksforGeeks. "What is a mashup in web technology?" [Internet]. Uppdaterad 2025-07-23. Hämtad 2026-03-18, från: <https://www.geeksforgeeks.org/websites-apps/what-is-a-mashup-in-web-technology/>
- [2] Vite. "Getting Started" [Internet]. Hämtad 2026-03-17, från: <https://vite.dev/guide/>
- [3] Netlify Docs. "Vite on Netlify" [Internet]. C 2026. Hämtad 2026-03-18, från: <https://docs.netlify.com/build/frameworks/framework-setup-guides/vite/>
- [4] NodeJs. "An introduction to the npm package manager" [Internet]. Hämtad 2026-03-17, från: <https://nodejs.org/en/learn/getting-started/an-introduction-to-the-npm-package-manager>
- [5] Mozilla. "CSS preprocessor" [Internet]. C 2026. Hämtad 2026-03-17, från: https://developer.mozilla.org/en-US/docs/Glossary/CSS_preprocessor
- [6] Sass. "Sass Basics" [Internet]. C 2026. Hämtad 2026-03-18, från: <https://sass-lang.com/guide/>
- [7] NPM. "Vite Image Optimizer" [Internet]. Hämtad 2026-03-17, från: <https://www.npmjs.com/package/vite-plugin-image-optimizer>
- [8] Maddumage R. "A Comprehensive Guide to JSDoc Comments in JavaScript" [Internet]. 2024. Hämtad 2026-03-17, från: <https://medium.com/@uomroshan/a-comprehensive-guide-to-jsdoc-comments-in-javascript-ed14df2351ef>
- [9] NPM. "JSDoc" [Internet]. Hämtad 2026-03-17, från: <https://www.npmjs.com/package/jsdoc>
- [10] JavaScript Info. "Error handling, 'try...catch'" [Internet]. C 2026. Uppdaterad 2025-04-08. Hämtad 2026-03-18, från: <https://javascript.info/try-catch>
- [11] JavaScript Info. "Promise" [Internet]. C 2026. Uppdaterad 2024-12-11. Hämtad 2026-03-18, från: <https://javascript.info/promise-basics>
- [12] W3Schools. "CSS Transitions" [Internet]. C 2026. Hämtad 2026-03-18, från: https://www.w3schools.com/css/css3_transitions.asp

Sundsvalls badplatser - Att skapa en mashup

Josefine Backlund 2026-03

- [13] W3Schools. "CSS Transform Property" [Internet]. C 2026. Hämtad 2026-03-18, från:
https://www.w3schools.com/cssref/css3_pr_transform.php
- [14] W3Schools. "CSS Animations" [Internet] C 2026. Hämtad 2026-03-18, från:
https://www.w3schools.com/css/css3_animations.asp
- [15] W3Schools. "CSS Animation Properties" [Internet]. C 2026. Hämtad 2026-03-18, från:
https://www.w3schools.com/css/css3_animations_properties.asp
- [16] Leaflet. "an open-source JavaScript library for mobile-friendly interactive maps" [Internet]. C 2025. Hämtad 2026-03-18, från:
<https://leafletjs.com/index.html>

Bilaga A: Figma-skiss

Mobil



Bilaga B: Figma-skiss

Desktop

